

Zygote v0.1

Six-Week Execution Plan

From specification to LinkedIn launch

Draft: May 15, 2026

Target launch: 2026-06-26

1 Overview

This document defines a six-week execution plan for shipping the first publicly demonstrable version of Zygote, beginning with specification review and ending with a LinkedIn announcement supported by a working demonstration. The plan assumes five hours of focused work per day, six days per week, throughout the summer of 2026, executed in parallel with paper-writing and internship recruiting at reduced intensity.

The plan is organized into three phases. Phase 0 (one day) is a complete review and revision of the existing specification document, performed by the author alone before any code is written. Phase 1 (two days) is a focused technical onboarding designed to remove the largest predictable sources of friction before development begins. Phase 2 (six weeks) is the development sprint itself, structured around weekly deliverables that build toward a single concrete launch artifact: a short screencast of the core fork-and-restore interaction.

Launch criterion. Phase 2 is complete when the author can record a 30–60 second GIF demonstrating: (1) creation of a node, (2) commit and execution against a real file, (3) fork of the resulting node into a new branch, (4) automatic restoration of workspace files to the pre-fork state, and (5) independent evolution of the new branch. No other feature is required for the LinkedIn launch.

2 Phase 0: Specification Review

Objective

Take complete ownership of the specification by reading it end-to-end, identifying every section where occupying decisions were made by Claude on the author's behalf, and replacing them with the author's own judgment.

Duration

One session, 2–2.5 hours, completed in a single sitting.

Method: Four-Pass Review

The specification should be reviewed in four sequential passes, not interactively edited section by section. Interactive editing dilutes ownership; the author must form a complete impression before changing anything.

1. **Pure reading** (30 min). Read the entire specification without modification. Close the document. Attempt to explain Zygote to an imagined stranger from memory. If unable, read again.
2. **Annotation only** (30 min). Reopen the document. Mark every passage requiring revision using inline comments. Do not yet write replacement text. This pass converts feeling into concrete action items.
3. **Revision** (45 min). Process all annotations. Write replacement text. Resolve all open marks.
4. **AI-assisted audit** (15 min). Submit the revised specification to a coding assistant with the following request: *(a) consistency check between the scope section and the demo scenario, (b) terminology audit confirming consistent usage of core terms, (c) identification of undeclared library or API dependencies in the technical architecture, (d) marking (not deletion) of redundant or compressible passages.* This is the only AI involvement permitted during Phase 0.

Mandatory Revision Targets

The following sections contain placeholder decisions and must be rewritten by the author:

- **§1 What is Zygote (opening paragraph).** The current draft is technically correct but tonally neutral. Rewrite using the author’s own voice. The two-sentence summary developed during this rewrite will be reused for the LinkedIn post, the GitHub repository description, and the blog header.
- **§3 ordering of subsections.** The current order is node → branch → chat → file state. Reconsider which of these is the most defensible differentiator. The first subsection in §3 becomes the product’s primary value claim by default.
- **§5.1 naming of the three interaction modes.** Current names: New Task, Node Chat, Floating Chat. Evaluate whether “Node Chat” remains the right term given that it explicitly demotes chat from a primary interface. Alternatives to consider include Node Exploration, Pre-commit Discussion, or other formulations.
- **§6.3 success criteria.** Four criteria are listed. For each, decide whether it reflects the author’s actual definition of success or whether it was inserted as a generic placeholder. Replace any that do not.

- **§9 roadmap.** Versions v0.2 through v0.5 are speculative. Rewrite to reflect the author’s actual intended evolution. If uncertain, delete v0.2–v0.5 entirely and leave only v0.1. An empty roadmap is more honest than a fabricated one.
- **§11 demo scenario.** The current scenario (JWT refactor, middleware versus decorator) was constructed for illustration. Replace with a real coding situation the author has experienced in the past two weeks. Authentic scenarios are substantially more persuasive than fabricated ones.

Sections to Read Without Necessarily Revising

The following sections do not require rewriting but must be understood well enough to recite without reference during development:

- §4 Information Architecture (four-level hierarchy)
- §5.2 Node Lifecycle (uncommitted → running → committed)
- §5.3 Fork Semantics, with particular attention to the choice not to copy a node’s internal chat into the forked branch
- §7.2 Data Model (TypeScript types serve as the operational schema)
- §7.5 Webview–Extension Host Protocol (the message contract)

Sections That May Be Deferred to Implementation

§2 (Problem), §7.1 (System Overview), §7.3 (Persistence Layout), §7.4 (Agent Runtime), §7.6 (File System Interaction), §7.7 (Project Structure), §8 (Competitive Differentiation), and §10 (Open Questions). These describe technical conventions and can be consulted as needed during development.

Anti-pattern: Specification perfectionism. The objective of Phase 0 is to make the specification good enough to begin coding, not to make it perfect. If the review extends past three hours, stop. Begin Phase 1. The specification will continue to evolve during implementation and the document at the end of v0.1 will look meaningfully different from the document at the start. This is expected.

3 Phase 1: Technical Onboarding

Objective

Eliminate the largest predictable sources of friction before the development sprint begins. Two days of focused onboarding is expected to save fifteen to twenty hours of unproductive debugging during weeks 1 and 2 of Phase 2.

Duration

Two days, five hours per day, ten hours total.

Rationale

“Learn while building” is sound advice for technologies with abundant documentation, immediate error feedback, and well-understood failure modes: TypeScript syntax, React component patterns, Claude API calls, file I/O. These can be addressed in real time without disrupting development.

It is unsound advice for technologies with non-obvious mental models and silent failure modes: the VS Code Extension API, the webview content security policy, and the message-passing protocol between the extension host and the webview. A debugging session in an unfamiliar webview frequently produces no error message; the user is left with a blank panel and no indication of what is wrong. Without prior conceptual understanding, this consumes entire evenings.

The onboarding phase exists to convert the second category into the first.

Day 1: Foundations

Time	Activity
0:00–1:30	Complete the official “Your First Extension” tutorial from the VS Code documentation. Build the resulting extension. Verify activation, command registration, and the debug launch configuration.
1:30–2:30	Locate a VS Code + webview + React template on GitHub with reasonable activity and clone it. Verify that the template builds and that the webview renders in a development host instance.
2:30–3:30	Modify the template to add a single button in the webview. Wire the button to send a message to the extension host. Have the extension host log the message to its output channel. Confirm the round-trip works.
3:30–4:30	Add a Claude API call to the extension host. Have it triggered by the webview button. Display the response in the webview. Use a hard-coded prompt. Confirm streaming or non-streaming response handling, whichever is simpler.
4:30–5:00	Configure the development environment: <code>launch.json</code> for debugging, <code>tasks.json</code> for build, environment variable handling for the API key.

Day 2: Concept Solidification

Time	Activity
0:00–1:00	Read the VS Code webview API documentation, focusing on the content security policy section and the message-passing API.
1:00–2:30	Read the VS Code workspace API documentation, focusing on <code>workspace.fs</code> for file system operations and <code>workspace.applyEdit</code> for editor modifications.
2:30–4:00	Modify the template to read a file from the workspace, display its contents in the webview, allow modification through a textarea, and write the modified contents back. Confirm that the editor pane reflects the change.
4:00–5:00	Sketch the directory structure for Zygote (as in spec §7.7). Create the empty folders. Move template code into the new structure. Commit to a fresh git repository.

Exit Criteria for Phase 1

Phase 1 is complete when all of the following are demonstrably working in a fresh extension host instance launched via F5:

1. A webview renders inside VS Code.
2. A button in the webview triggers an extension host action.
3. The extension host calls the Claude API and returns a response to the webview.
4. The extension host reads and writes files in the user’s workspace.
5. A git repository exists with the directory structure ready for Zygote development.

4 Phase 2: Six-Week Development Sprint

Structure

The sprint is organized into six weeks, each with a single demonstrable deliverable. The deliverable defines whether the week is complete; partial progress without a demonstrable artifact counts as incomplete. This structure exists to convert the open-ended task of “build Zygote” into a sequence of concrete, falsifiable claims.

Week 1: Tree Persistence

Deliverable. A working extension that allows the user to type a prompt, sends the prompt to the Claude API, displays the response, and persists the prompt-response pair as a node in a tree stored at `.zygote/tree.json`. Closing and reopening VS Code restores the tree.

Scope. Data model implementation (TypeScript types from spec §7.2). Persistence layer (read and write `tree.json`). Basic webview UI displaying the tree as an indented list. Single-node creation. No internal chat, no commit/uncommit states, no fork, no tool use.

Out of scope this week. Every other feature.

Week 2: Node Lifecycle

Deliverable. Nodes have three states (uncommitted, running, committed). The user can create an uncommitted node, edit its prompt, conduct an internal chat with the agent without committing, and finally commit. The commit triggers a final agent response that becomes the node's permanent output.

Scope. The state machine for node lifecycle. Internal chat UI inside the selected node's detail panel. Commit button. Streaming or polling updates from extension host to webview during execution.

Out of scope this week. Tool use, file modification, fork, snapshot, floating chat.

Week 3: Tool Use and File Modification

Deliverable. The committed agent can read files from the workspace and write files back. The user can observe file changes in the editor pane as the agent executes. Each committed node records the tool calls it made and the files it modified.

Scope. Tool definitions for `read_file` and `write_file`. Tool execution loop in the extension host. Tool call results recorded into the node's data structure. The editor pane reflects file changes automatically via VS Code's file watching.

Out of scope this week. Shell command execution. Multi-turn agent loops within a single node. Snapshot system.

Week 4: Snapshot and Fork

Deliverable. Each committed node stores a content-addressable snapshot of the files it modified. The user can fork any committed node. Forking creates a new branch, restores the workspace files to the pre-node state, and switches the active branch.

Scope. Content-addressable storage in `.zygote/snapshots/`. Snapshot capture on commit. Snapshot restoration on fork. Branch data model. Active branch tracking. Branch switching restores files.

Out of scope this week. Branch comparison UI, merge, multi-turn agent loops.

Week 4 is the structural midpoint of the project. The fork-and-restore mechanism is the core of the LinkedIn demonstration. By the end of this week, the central artifact of the launch is in principle producible. The remaining two weeks polish presentation rather than add capability.

Week 5: Polish and Floating Chat

Deliverable. The tree visualization is legible (proper indentation, status indicators, selection highlighting). Branch switching is accessible via a dropdown. The floating chat is implemented as a secondary panel. The detail pane formatting is consistent.

Scope. UI refinement throughout. Floating chat implementation (ephemeral, not persisted). Minor bug fixes accumulated from weeks 1–4.

Out of scope this week. New core features.

Week 6: Recording and Launch

Deliverable. A 30–60 second screencast demonstrating the core loop. A GitHub repository with a README explaining setup. A LinkedIn post containing the screencast and a single-sentence framing of the product.

Scope. Demo rehearsal. Screencast recording (likely several takes). README writing. GitHub repository preparation. LinkedIn post drafting. The post itself goes live at the end of week 6.

Out of scope this week. Code changes beyond fixing recording-blocking bugs.

Weekly Deliverable Summary

Wk	Theme	Demonstrable artifact
1	Tree Persistence	Single-node tree, persisted across restarts
2	Lifecycle	Uncommit/commit states with internal chat
3	Tool Use	Agent reads and writes real files
4	Fork	File system restoration on fork
5	Polish	Refined UI, floating chat
6	Launch	Public LinkedIn post with screencast

5 Daily Schedule

Time Allocation

The plan assumes the following daily schedule during Phase 2, scaled to the summer-with-paper-and-internship-search context:

Activity	Daily	Notes
Zygote development	5h	Two blocks of 2.5h, separated by at least one hour
Paper writing	1.5–2h	Single morning or evening block
Internship search	0.5–1h	Application submissions, recruiter follow-up
Exercise	1–1.5h	Existing PPL/football/run schedule
Sleep	7–8h	Non-negotiable

Recommended Daily Structure

The five hours of Zygote development should be split into two blocks rather than a single five-hour block. Sustained focus on novel technical work degrades after roughly 2.5 hours; a break preserves quality for the second block.

Window	Activity
Morning (9:00–12:00)	Zygote Block A: 2.5h of focused implementation. The harder problem of the day goes here.
Midday (12:00–14:00)	Lunch, paper writing, or rest.
Afternoon (14:00–16:30)	Zygote Block B: 2.5h. Lower-risk tasks: UI refinement, testing, refactoring.
Late afternoon (16:30–18:00)	Exercise.
Evening (18:00–21:00)	Paper writing, internship search, social time.
Pre-sleep (21:00–22:30)	Reading, dev log writing, planning the next day. Never coding.

Weekly Rhythm

Six days of development, one day of rest. Recommended rest day: Sunday. The rest day is not optional; sustained six-day weeks without recovery produce code quality degradation by the end of week 3 even when the developer subjectively feels fine.

6 Discipline and Guardrails

The plan depends on three commitments. Without these the schedule degrades to its worst-case duration regardless of total hours worked.

Commitment 1: Calendared Time Blocks

Both Zygote blocks are scheduled in advance and treated as fixed appointments. “9:00–12:00 Tuesday” is a meeting, not a flexible intention. The author’s existing push-pull-legs training schedule demonstrates capacity for fixed-time discipline; the same standard applies here.

Commitment 2: One Weekly Demonstrable Artifact

Every week ends with something the author can demonstrate to another person in under two minutes. A week without a demonstrable artifact counts as zero progress, regardless of hours spent or code written. This converts the open-ended task into a sequence of falsifiable checkpoints.

Commitment 3: Weekly Development Log

At the end of each week, the author writes a 200-word development log: what was completed, what blocked progress, what is planned for the following week. The log is private during Phase 2 but constitutes the primary source material for the launch blog post and any subsequent reflective writing. Documentation written during development is significantly more accurate than documentation reconstructed afterward.

7 Risks and Mitigations

Risk 1: Webview rendering failures consume multiple days

The webview content security policy frequently blocks scripts, styles, or resources without producing useful error messages. The author may spend a full day attempting to render a React application inside the webview and obtain no visible output.

Mitigation. Phase 1 Day 1 explicitly addresses this by starting from a working webview-and-React template. The template is treated as canonical and not modified beyond what is necessary until full understanding of the CSP is achieved.

Risk 2: Tool use complexity underestimated

Week 3 is allocated for tool use implementation. This is the technically densest week of the plan: the agent loop must handle multi-step tool sequences, errors, and incremental output streaming back to the webview. Five days may not be sufficient.

Mitigation. Week 3 deliverables are scoped to two tools only (`read_file`, `write_file`), single-turn execution, and no streaming requirement. Multi-turn loops and additional tools are deferred to v0.2.

Risk 3: Scope creep during weeks 5–6

As week 6 approaches, the author may attempt to add features that “would make the demo better.” Each additional feature postpones the launch by an indeterminate amount.

Mitigation. Week 4 is the final week in which new features are added. Weeks 5 and 6 are reserved for polish and launch preparation respectively. Any feature not present at the end of week 4 is automatically deferred to v0.2.

Risk 4: Parallel commitments degrade development quality

Five hours of Zygote development, two hours of paper writing, and an hour of internship search per day is approximately eighteen total hours of cognitively demanding work, against a sustainable budget of approximately twelve hours for most individuals.

Mitigation. The plan acknowledges this and assigns paper and internship work to lower-quality time blocks (evenings, late afternoons). If quality degradation appears—measured by missed weekly deliverables, recurring small mistakes, or self-reported exhaustion—reduce paper writing to one hour daily and accept paper delay.

Risk 5: External release of competing functionality

Anthropic’s Agent View is currently in research preview. If Anthropic or another vendor ships tree visualization or fork-restoration functionality before the v0.1 launch, the perceived novelty of Zygote diminishes.

Mitigation. The launch criterion is intentionally narrow: file-state-bound fork is the demonstrated capability. This is genuinely absent from current public products and is unlikely to be shipped by a major vendor within six weeks. If a competing product launches during weeks 1–5, the

response is to ship sooner with a sharper differentiation message rather than to extend the timeline.

8 Launch Plan

Day of Launch

At the end of week 6, the LinkedIn post goes live. The post contains the screencast, a single-sentence framing of the product, and a link to the GitHub repository. It does not contain explanatory paragraphs, requests for feedback, or extensive technical detail. The post’s purpose is timestamp establishment and curiosity generation, not conversion.

Two Weeks After Launch

A longer blog post is published, expanding on the LinkedIn announcement. The blog post is the appropriate venue for the philosophical framing (conversation as wrong unit of work, node as commitment, file state as evolution of thinking). Distribution channels include Hacker News, X, and the author’s existing WeChat audience (in translation).

Four Weeks After Launch

A reflective piece is published: lessons from building v0.1, what surprised the author, what failed. This piece is the foundation for the developer’s identity in this space transitioning from “person who built a tool” to “person with judgment about agent UX.” The latter identity is what startup recruiters and investors respond to.

Eight Weeks After Launch

A response piece commenting on recent ecosystem developments (Anthropic releases, competing tools, Cursor updates) framed through Zygote’s value proposition. By this point the author is participating in industry conversation as a recognized voice rather than as an unknown developer.

Appendix A: Summer Parallel Workload

The plan operates against three concurrent summer commitments. The relative priority and time allocation is as follows:

Commitment	Hours/day	Status at summer’s end
Zygote v0.1	5	Launched publicly with screencast and blog
Paper writing	1.5–2	Complete first draft, ready for advisor review
Internship search	0.5–1	Active applications submitted, recruiter pipeline established

The intentional asymmetry reflects the plan’s hypothesis that Zygote has the highest variance of outcomes among the three: a successful launch produces compounding career returns over months and years, while a delayed paper or a shifted internship cycle produces linear cost. The asymmetry is correct only if the launch actually occurs. A delayed Zygote launch combined with reduced effort on paper and internship work is the worst outcome and must be actively prevented.

Appendix B: Day-One Checklist

The author begins execution on the first business day following completion of this plan. Day one consists of:

1. Open the specification document.
2. Execute Phase 0 in a single 2–2.5 hour session.
3. Submit revised specification for AI-assisted audit.
4. Process audit suggestions.
5. Schedule Phase 1 days into the calendar.
6. Write a 100-word commitment statement describing what the author expects to be true six weeks from today.

The commitment statement is read at the start of each week during Phase 2.

End of plan.